ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO HRI

BÀI THỰC HÀNH 05

TURTLEBOT 4 LOCALIZATION AUTONOMOUS NAVIGATION & YOLO

Họ và tên sinh viên: Đỗ Việt Anh

Mã sinh viên: 23020719

Lớp: K68E-RE

Giảng viên hướng dẫn: Kỹ sư Nguyễn Quốc Bảo

Mã nguồn GitHub: turtlebot4_bai05

Video demo: Google Drive

Mục lục

1 Tổng quan công việc tuần 5 **1**

2 Yêu cầu bài toán và môi trường sử dụng **1**

2.1 Yêu cầu cần đáp ứng . . . 1

2.2 Các thành phần phần mềm . . . 1

3 Cấu trúc hệ thống và luồng hoạt động **2**

3.1 Cấu trúc thư mục . . . 2

3.2 Sơ đồ khối các thành phần . . . 4

3.3 Các node và thành phần chính . . . 4

3.4 Topic, action và hệ tọa độ . . . 4

3.5 Luồng hoạt động khi chạy . . . 6

4 Môi trường mô phỏng và nhiệm vụ thực hành **6**

4.1 Bố trí môi trường . . . 6

4.2 Nhiệm vụ . . . 7

5 Xây dựng bản đồ bằng SLAM Toolbox **7**

5.1 Nguyên lý và thao tác mapping . . . 7

5.2 Bản đồ đã lưu . . . 8

6 Localization sau khi nạp lại bản đồ **8**

6.1 Trình tự khởi tạo . . . 8

6.2 Quan sát localization và đối chiếu log . . . 9

7 Navigation với Nav2 **9**

7.1 Lập kế hoạch và điều khiển . . . 9

7.2 Ảnh RViz . . . 10

7.3 Thực hiện và phân loại goal . . . 10

7.4 Nhận diện vật thể từ camera bằng YOLO . . . 11

8 Khó khăn gặp phải và cách xử lý **11**

8.1 Bridge sử dụng sai tên world . . . 11

8.2 Tham số ROS và đường dẫn plugin mô phỏng . . . 11

8.3 Mô phỏng chậm trên màn hình ảo . . . 12

8.4 Giới hạn quan sát và nhận diện . . . 12

9 Kết quả đạt được **12**

9.1 Mô phỏng, sensor và map . . . 12

9.2 Kết quả từng navigation goal . . . 12

9.3 Quãng đường, thời gian và sai lệch . . . 13

9.4 Quỹ đạo AMCL theo từng goal . . . 14

9.5 Quãng đường tích lũy theo thời gian . . . 15

9.6 Kết quả nhận diện và đánh giá . . . 16

10 Cách chạy code **16**

10.1 Chạy all bằng một lệnh . . . 16

11 Kiến thức và kỹ năng đã học được **17**

11.1 Phân biệt mapping, localization và navigation . . . 17

11.2 Làm việc với hệ tọa độ và thời gian ROS 2 . . . 17

11.3 Nhận biết giới hạn của sensor . . . 17

11.4 Lập trình node và đánh giá kết quả . . . 17

Tài liệu tham khảo **17**

1 Tổng quan công việc tuần 5

Bài thực hành 05 sử dụng lại bản đồ môi trường maze đã lưu ở tuần 4 để định vị TurtleBot 4 bằng AMCL, điều khiển robot tự động qua ba điểm mục tiêu bằng code và nhận diện vật thể từ camera RGB bằng YOLO. Công việc được triển khai theo trình tự: khởi chạy đúng world; nạp map; cho robot ra khỏi dock và khởi tạo pose; chờ AMCL/Nav2 sẵn sàng; chạy node điều hướng và node nhận diện đồng thời; ghi kết quả để đánh giá.

So với tuần 4 đặt goal bằng RViz, tuần này em đưa danh sách pose vào file cấu hình và dùng `nav2_simple_commander` để gửi lần lượt từng goal. Camera được xử lý ở node riêng nhằm tách vòng nhận ảnh/suy luận khỏi chương trình gửi action. Bản cập nhật ngày 20/10/2026 sử dụng số liệu và ảnh camera từ lượt `direct_goal_route_run10`, chạy source commit `c87263f`. Các lượt cũ, gồm run04, được giữ lại để đối chiếu.

2 Yêu cầu bài toán và môi trường sử dụng

2.1 Yêu cầu cần đáp ứng

Robot phải nạp được hai file `map.yaml`, `map.pgm` của tuần trước trong cùng world maze. Sau khi thiết lập Initial Pose, robot và dữ liệu `/scan` phải tương đối khớp bản đồ trên RViz2. Chương trình gửi ít nhất ba goal thuộc các khu vực khác nhau trong frame `map`, chờ kết quả goal hiện tại rồi mới chuyển sang goal tiếp theo. Trong lúc chạy không phát teleop và không đặt Nav2 Goal bằng chuột.

Node YOLO subscribe ảnh RGB, chuyển Image message sang OpenCV bằng `cv_bridge`, suy luận mô hình, vẽ bounding box kèm class/confidence và in thông tin ra terminal. Video cần thể hiện navigation hoạt động cùng cửa sổ nhận diện.

2.2 Các thành phần phần mềm

Thành phần	Môi trường sử dụng	Vai trò
Hệ điều hành	Ubuntu 22.04	Chạy trực tiếp trên máy Linux.
ROS 2	Humble	Node, topic, action, TF và simulation time.
Mô phỏng	Ignition Gazebo Fortress	World maze, TurtleBot 4 Standard và sensor.
Localization	MapServer + AMCL	Nạp map, ước lượng pose và TF.
Navigation	Nav2 + Simple Commander	Navfn global planner, DWB local controller.
Nhận diện	Ultralytics, YOLO-World TorchScript	Suy luận ảnh RGB với confidence 0,12; imgsz=960.
Xử lý ảnh	OpenCV 4.11.0, cv_bridge	Chuyển ảnh, annotate và hiển thị.
Tính toán	PyTorch 2.11.0+cu130, CUDA	Chạy YOLO trên GPU khi khả dụng.
Hiển thị	RViz2, Gazebo GUI, OpenCV	Map, scan, pose, path và detection.

Bảng 1: *Phần mềm được sử dụng trong Bài 05.*

3 Cấu trúc hệ thống và luồng hoạt động

3.1 Cấu trúc thư mục

Các file chính dùng trong chương trình được bố trí như sau:

```
bai05_turtlebot4/
|-- launch/demo.launch.py
|-- scripts/
|   |-- run_demo.sh
|   |-- arrange_windows.py
|   |-- navigate_goals.py
|   `-- yolo_detector.py
|-- config/
|   |-- goals.yaml
|   |-- localization.yaml
|   |-- nav2.yaml
|   `-- gazebo_gui.config
|-- maps/map.yaml, map.pgm
|-- worlds/maze_yolo.sdf, placements.yaml
|-- models/
|   |-- malevisitoronphone/
|   |-- fire_extinguisher/
|   `-- yolo_world_furniture.torchscript
|-- rviz/navigation.rviz
|-- docs/
`-- package.xml, CMakeLists.txt, README.md, LICENSE, NOTICE
```

File khởi chạy `launch/demo.launch.py`. File này ghép các thành phần cần dùng vào một lệnh launch: Gazebo, robot TurtleBot 4, bridge clock, MapServer/AMCL, Nav2, RViz và hai node Python của bài tập. Các thành phần được gọi theo thời gian trễ để mô phỏng và cảm biến có thời gian khởi tạo trước khi chương trình gửi goal. Launch đọc các file trong `config/`, chọn world trong `worlds/` và truyền đường dẫn model YOLO cho node nhận diện. Các tham số như `run_route`, `output_dir`, `device` và `yolo_model` cho phép chọn cách chạy mà không cần sửa nội dung hai node.

Hai script hỗ trợ chạy demo. `scripts/run_demo.sh` là file người dùng chạy từ terminal. Script nạp môi trường ROS 2, build các package cần thiết, tạo thư mục lưu kết quả rồi gọi `demo.launch.py`. Script cũng chọn ROS domain và Ignition partition riêng cho từng phiên. `scripts/arrange_windows.py` dùng xdotool để đặt RViz ở bên trái, Gazebo ở góc trên bên phải và cửa sổ YOLO ở góc dưới bên phải. File này chỉ căn cửa sổ để quan sát/quay video, không tham gia điều khiển robot.

Node điều hướng `scripts/navigate_goals.py`. File định nghĩa lớp `RouteNavigator` kế thừa `BasicNavigator`. Node đọc danh sách đích từ `goals.yaml`, kiểm tra trạng thái dock, thực hiện Undock và khởi tạo AMCL trong mô phỏng. Với mỗi đích, node tạo `PoseStamped`, chuyển yaw thành quaternion, gọi `goToPose()` rồi chờ action kết thúc. Sau đó node kiểm tra kết quả và các bộ đếm va chạm trước khi gửi goal tiếp theo. Node đồng thời nhận pose AMCL, odometry, ground truth và hazard để lưu `trajectory.csv`, `navigation_events.jsonl` và `goal_results.json`. Các marker tên Goal 1–3 trên RViz cũng được phát từ file này.

Node nhận diện `scripts/yolo_detector.py`. Node nhận ảnh trên `/oakd/rgb/preview/image_raw`, dùng `cv_bridge` chuyển ảnh ROS sang ảnh BGR của OpenCV rồi đưa vào YOLO. Bản demo truyền model YOLO-World TorchScript từ launch. Node lấy class, confidence và tọa độ box, giữ hai lớp `person`, `fireextinguisher` để vẽ lên ảnh. Kết quả được hiển thị, publish lại thành Image/JSON và lưu cùng ảnh gốc để đối chiếu. Node còn nhận trạng thái nhiệm vụ để ghi dòng chữ robot đang đi tới goal nào; kết quả YOLO không phát lệnh vận tốc hoặc gửi thêm navigation goal.

Cấu hình đích trong `config/goals.yaml`. File lưu frame của đích, ba bộ tọa độ x, y, yaw, timeout cho mỗi goal và thời gian dừng giữa hai goal. Bản hiện tại dùng frame `map`, timeout 600 giây và dừng 6 giây. Muốn đổi điểm đến hoặc hướng robot khi dừng thì sửa file này; không cần thay vòng lặp điều hướng trong Python.

Cấu hình định vị và điều hướng. `config/localization.yaml` chứa tham số MapServer và AMCL: frame map/odom/thân robot, topic scan, mô hình chuyển động và số hạt. Ví dụ, AMCL hiện đặt số hạt trong khoảng 500–2.000. `config/nav2.yaml` cấu hình planner Navfn, controller DWB, costmap và các hành vi hỗ trợ. Những giá trị như tốc độ thẳng tối đa 0,26 m/s, bán kính inflation 0,45 m và sai số vị trí goal checker 0,25 m nằm trong file này. Hai file có vai trò khác nhau: AMCL ước lượng robot đang ở đâu, còn Nav2 dùng pose và costmap để tìm đường rồi tính vận tốc.

Bản đồ trong `maps/`. `map.pgm` là ảnh occupancy grid lưu các ô tường, vùng trống và vùng chưa biết. `map.yaml` chỉ đến ảnh đó, đồng thời lưu độ phân giải 0,05 m/ô, origin và các ngưỡng chuyển mức xám thành occupied/free. Hai file phải đi cùng nhau để MapServer nạp đúng tọa độ. Đây là bản sao nguyên byte của map tuần 4; launch mặc định lấy map từ package `bai04_turtlebot4`. Những bình chữa cháy thêm vào mô phỏng không được vẽ lại trong cặp file map này.

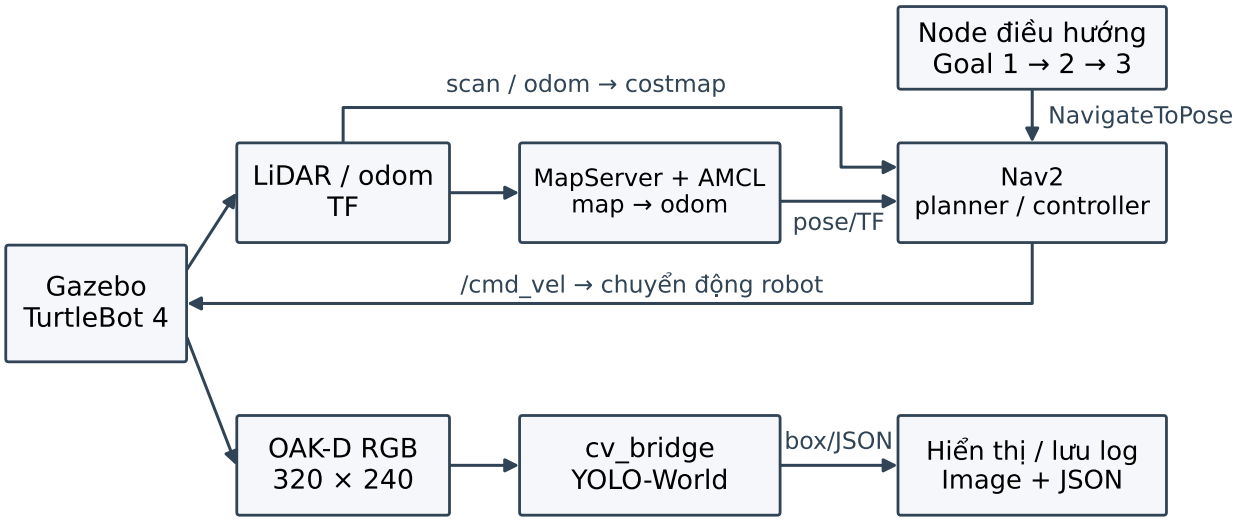
World và vị trí vật thể trong `worlds/`. `maze_yolo.sdf` mô tả tường maze và các model được đặt trong Gazebo. Các pose trong file SDF quyết định vị trí thực tế của một người và hai bình chữa cháy. `placements.yaml` ghi lại tên model, tọa độ và yaw để dễ kiểm tra bố trí; launch hiện nạp SDF, không tự sinh vị trí vật thể từ file YAML này. Vì vậy, khi đổi scene cần sửa pose trong SDF và cập nhật bảng vị trí tương ứng.

Tài nguyên trong `models/`. `malevisitoronphone/` chứa hình học, vật liệu và thông tin nguồn của model người. `fire_extinguisher/` chứa `model.sdf` dựng bình màu đỏ, nhãn trắng, tay cầm, ống và phần collision để mô phỏng vật cản. File `model.config` mô tả tên model, phiên bản và file SDF cần nạp. `yolo_world_furniture.torchscript` là model nhận diện được nạp vào node YOLO; file này xử lý ảnh, khác với hai thư mục model 3D dùng để dựng vật thể trong Gazebo.

Cấu hình hiển thị. `rviz/navigation.rviz` lưu Fixed Frame, góc nhìn và các display Map, LaserScan, costmap, path, marker goal, camera OAK-D RGB. Nhờ đó RViz mở ra với đúng các lớp cần quan sát thay vì phải thêm lại từng display. `config/gazebo_gui.config` lưu cấu hình GUI/góc nhìn của Gazebo, không thay đổi bản đồ hay thuật toán Nav2.

Các file khai báo và tài liệu. `package.xml` khai báo tên package và các phụ thuộc ROS như rclpy, nav2_simple_commander, cv_bridge và ros_gz_bridge. `CMakeLists.txt` quy định cách cài đặt hai node vào thư mục executable của package và đưa launch, config, map, world, model vào thư mục share khi build. `README.md` ghi cách cài, build và chạy; `LICENSE`, `NOTICE` ghi giấy phép và nguồn tài nguyên. Thư mục `docs/` chứa PDF, LaTeX, hình, log run10 và script dựng báo cáo. Kết quả của một lần chạy mới được ghi vào thư mục `output_dir` do người dùng chọn, gồm log điều hướng, CSV quỹ đạo, JSON detection và ảnh camera.

3.2 Sơ đồ khối các thành phần



Hình 1: *Luồng dữ liệu Localization, Navigation và YOLO. Hai node bài tập sử dụng chung dữ liệu mô phỏng nhưng chạy ở hai process.*

3.3 Các node và thành phần chính

Thành phần	Chức năng
Gazebo và ROS–Gazebo bridge	Mô phỏng robot/sensor; chuyển scan, ảnh, odometry và clock sang ROS.
robot_state_publisher	Xuất TF của thân robot và các sensor.
map_server / amcl	Phát occupancy map, cập nhật particle và TF *map* → *odom*.
planner_server / controller_server	Lập global path và tính lệnh vận tốc để bám path.
bt_navigator / behavior_server	Điều phối NavigateToPose và hành vi hỗ trợ.
lifecycle_manager	Configure/activate các node Nav2.
bai05_route_navigator	Undock, khởi tạo AMCL, gửi goal tuần tự, lưu action result và telemetry.
bai05_yolo_detector	Nhận Image, cv_bridge, YOLO, publish ảnh/JSON detection và hiển thị.

Bảng 2: *Chức năng các thành phần chính.*

3.4 Topic, action và hệ tọa độ

Topic được dùng để truyền dữ liệu liên tục như scan, ảnh, pose và vận tốc. Các yêu cầu kéo dài như đi đến một đích hoặc rời dock dùng action để bên gửi theo dõi tiến độ, nhận kết quả và có thể hủy yêu cầu đang thực hiện.

Bản đồ và dữ liệu LiDAR. `map_server` đọc cặp YAML/PGM rồi phát `/map`, kiểu `nav_msgs/msg/OccupancyGrid`. Message chứa kích thước lưới, độ phân giải, origin và giá trị từng ô; AMCL dùng nó để so khớp scan, static layer của costmap dùng nó để biết các tường đã có. `/scan`, kiểu `sensor_msgs/msg/LaserScan`, mang dãy khoảng cách LiDAR theo từng góc quét. Dữ liệu scan từ mô phỏng được đưa sang ROS qua bridge, sau đó AMCL và các lớp quan sát vật cản của Nav2 cùng sử dụng. Vì vậy bình mới có thể được phát hiện bằng LiDAR dù chưa xuất hiện trong map tĩnh.

Odometry và pose sau định vị. `/odom`, kiểu `nav_msgs/msg/Odometry`, mô tả chuyển động tương đối và vận tốc robot trong hệ odom. Nav2 dùng dữ liệu này khi điều khiển chuyển động; node điều

hướng cũng lưu vận tốc để đối chiếu quỹ đạo. Sau khi kết hợp map, scan và chuyển động, AMCL phát `/amcl_pose`, kiểu `geometry_msgs/msg/PoseWithCovarianceStamped`. Message chứa vị trí, hướng, timestamp và covariance biểu diễn độ bất định của ước lượng. Node điều hướng đọc pose này để ghi CSV và tính khoảng cách từ vị trí ước lượng cuối tới goal.

Pose khởi tạo và ground truth. `/initialpose` có cùng kiểu PoseWithCovarianceStamped, là đầu vào đặt ước lượng ban đầu cho AMCL. Trong mô phỏng, node lấy pose từ `/sim_ground_truth_pose` (kiểu Odometry) rồi gọi `setInitialPose()` của BasicNavigator để publish ước lượng đó. Ground truth còn được ghi cùng pose AMCL để đối chiếu. Đây là thông tin tham chiếu có sẵn trong Gazebo; khi áp dụng cho robot thật cần có cách xác định pose ban đầu tương ứng. Run10 không ghi nhận lần đặt lại pose giữa tuyến.

Action đi đến goal và rời dock. `/navigate_to_pose` dùng kiểu `nav2_msgs/action/NavigateToPose`. `bai05_route_navigator` là action client, còn bt_navigator của Nav2 nhận yêu cầu. Goal chứa một PoseStamped trong frame map; Nav2 cung cấp feedback về quá trình di chuyển và trả trạng thái khi nhiệm vụ kết thúc. Chương trình chờ action hiện tại hoàn tất, ghi kết quả rồi mới gửi pose kế tiếp. Riêng thao tác ra khỏi dock dùng `/undock`, kiểu `irobot_create_msgs/action/Undock`. Node đọc `/dock_status` (DockStatus) trước; chỉ gửi Undock khi robot đang ở dock và chờ thành công trước khi bắt đầu khởi tạo/điều hướng.

Lệnh vận tốc và tín hiệu va chạm. `/cmd_vel` có kiểu `geometry_msgs/msg/Twist`; hai thành phần chính dùng cho robot vi sai là `linear.x` (m/s) và `angular.z` (rad/s). Controller Nav2 tính lệnh bám đường; trong launch đang dùng, lệnh qua `/cmd_vel_nav` tới velocity smoother trước khi ra `/cmd_vel` để điều khiển robot mô phỏng. `/hazard_detection` có kiểu `irobot_create_msgs/msg/HazardDetectionVector`, còn `/bumper_contact` có kiểu `ros_gz_interfaces/msg/Contacts`. Node điều hướng đọc hai topic để đếm BUMP/contact và lưu event; các bộ đếm này được kiểm tra cùng action result trước khi cho phép chuyển sang goal tiếp theo.

Ảnh RGB và kết quả nhận diện. `/oakd/rgb/preview/image_raw` là `sensor_msgs/msg/Image`, do camera mô phỏng cung cấp qua bridge. Ảnh trong run10 có kích thước 320×240. RViz nhận topic này để hiển thị panel OAK-D RGB; node YOLO nhận cùng nguồn ảnh với QoS best effort, hàng đợi một message để ưu tiên khung mới. Sau suy luận, node publish `/bai05/yolo/image` dưới dạng Image đã vẽ box, đồng thời phát `/bai05/yolo/detections` dưới dạng `std_msgs/msg/String` chứa JSON. JSON ghi timestamp ảnh, class, confidence và box `[x1,y1,x2,y2]` theo pixel. Các tọa độ này thuộc ảnh camera, chưa phải tọa độ vật thể trên map và không được dùng để gửi goal hay thay đổi lệnh điều khiển Nav2.

Trạng thái nhiệm vụ và marker. `/bai05/mission_status` là String chứa event JSON do node điều hướng phát, ví dụ GOAL_SENT, GOAL_RESULT và MISSION_COMPLETE. Node YOLO đọc topic này để đổi dòng trạng thái trên ảnh. `/waypoints`, kiểu `visualization_msgs/msg/MarkerArray`, chứa các nhãn Goal 1–3 trong frame map để RViz vẽ tại vị trí đích. Marker chỉ phục vụ quan sát; action đi đến đích được gửi riêng qua NavigateToPose.

Quan hệ giữa các hệ tọa độ. `/tf` và `/tf_static` có kiểu `tf2_msgs/msg/TFMessage`, mô tả các phép biến đổi động và các quan hệ cố định giữa những frame. Chuỗi chính là

$$\texttt{map} \longrightarrow \texttt{odom} \longrightarrow \texttt{base_link} \longrightarrow \texttt{sensor}.$$

Frame map gắn với bản đồ đã lưu; AMCL cập nhật biến đổi map–odom để hiệu chỉnh ước lượng theo scan/map. Biến đổi odom–base_link mô tả chuyển động cục bộ của thân robot. Các biến đổi từ thân robot tới LiDAR/camera lấy từ mô hình robot, được robot_state_publisher phát để dữ liệu sensor có thể quy về frame chung. RViz chọn Fixed Frame = map, nhờ đó map, scan, robot và các goal được đặt đúng tương quan. Nếu thiếu TF hoặc dùng timestamp không tra được biến đổi, scan/path có thể không hiển thị đúng dù topic vẫn đang phát dữ liệu.

Mỗi goal trong bài có `header.frame_id=map`; x, y là tọa độ tính theo bản đồ và yaw là hướng cuối tính bằng radian. Với chuyển động trên mặt phẳng, node đặt quaternion $q_x = q_y = 0$, $q_z = \sin(yaw/2)$, $q_w = \cos(yaw/2)$. Timestamp của PoseStamped lấy từ clock ROS tại thời điểm gửi để Nav2 sử dụng cùng mốc thời gian với dữ liệu TF và cảm biến.

Clock mô phỏng và thời gian chờ. Bridge phát `/clock`, kiểu `rosgraph_msgs/msg/Clock`, từ đồng hồ Gazebo. Các node đặt `use_sim_time=true` nên timestamp pose/ảnh và các timer ROS theo thời gian mô phỏng. Chương trình còn dùng đồng hồ thực để giới hạn thời gian chờ một goal và dừng giữa hai goal. Vì vậy khi Gazebo chậm hoặc tạm dừng, thời gian mô phỏng có thể tăng chậm/dừng trong khi timeout thực vẫn tiếp tục. CSV lưu cả timestamp AMCL và ground truth để biết hai pose đem so sánh có được ghi nhận ở cùng thời điểm hay không.

3.5 Luồng hoạt động khi chạy

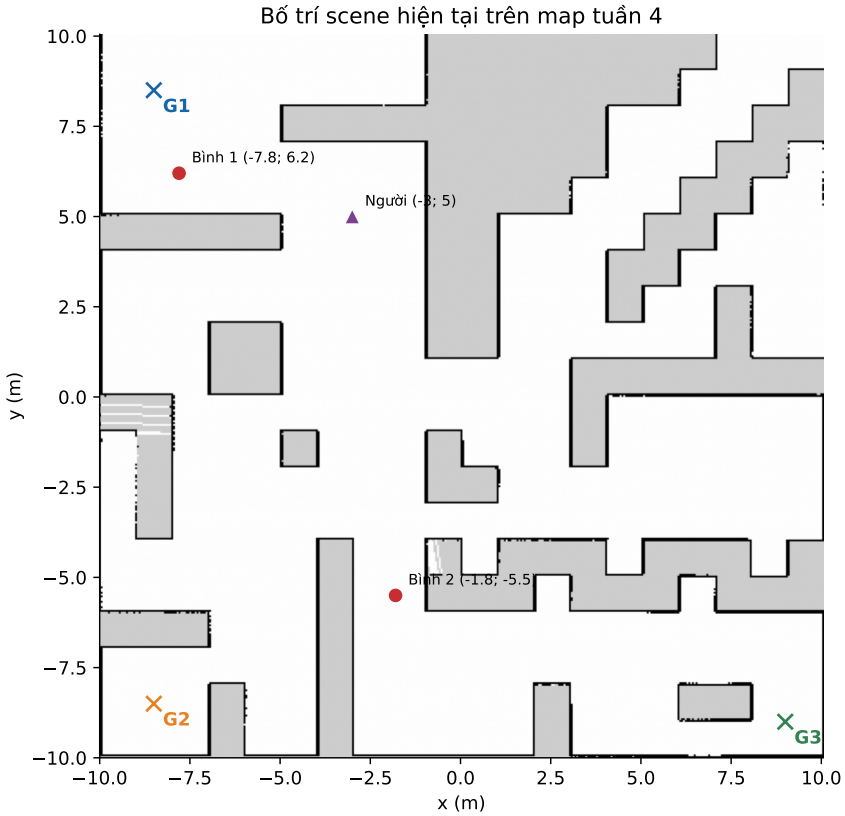
Launch mở mô phỏng và nạp map cùng cấu hình Nav2. Node navigation chờ clock, ground truth và dock status; nếu robot đang dock thì gửi action Undock và chờ thành công. Pose Gazebo sau undock được dùng làm ước lượng khởi tạo AMCL. Khi AMCL/Nav2 active, node đọc goals.yaml, gửi Goal 1, chờ action kết thúc, ghi kết quả, dừng khoảng 6 giây wall time rồi tiếp tục Goal 2 và Goal 3; không gửi pose quan sát chen giữa. Mỗi goal có timeout 600 giây wall time. Lượt thất bại, timeout hoặc có BUMP/contact làm nhiệm vụ dừng để kiểm tra. Không có lệnh teleop trong vòng lặp này.

Node YOLO nhận ảnh mới nhất bằng QoS best effort, depth 1. Callback subscriber chỉ giữ ảnh; timer 0,25 giây mô phỏng thực hiện suy luận. Cách này hạn chế xử lý hàng đợi ảnh cũ. Node publish ảnh annotate và JSON, lưu log, đồng thời mở cửa sổ OpenCV. Mission status được truyền qua topic để hiển thị goal đang chạy trên cửa sổ camera.

4 Môi trường mô phỏng và nhiệm vụ thực hành

4.1 Bố trí môi trường

World maze gồm tường, hành lang, ngã rẽ và các vùng mở như tuần trước. Robot bắt đầu tại gần tâm world cùng dock sạc. Scene cuối giữ một model MaleVisitorOnPhone tại $(-3; 5)$, một bình chữa cháy gần Goal 1 tại $(-7,8; 6,2)$ và một bình trên tuyến Goal 2 đến Goal 3 tại $(-1,8; -5,5)$. Em bỏ bàn, ghế và người ở vị trí chạm/xuyên tường. Hai bình là model hình học SDF có collision, người là model Gazebo Fuel. Node nhận diện nhận ảnh RGB thật từ camera mô phỏng.



Hình 2: *Sơ đồ bố trí scene cuối trên map tuần 4, lấy từ world SDF của run10. Ký hiệu người/bình là vị trí model trong mô phỏng; các vật thêm vào không sửa file map.*

Vị trí vật thể phụ thuộc bố trí tường nên không phải lúc nào camera cũng nhìn thấy. Tại một số hướng, tường che phần lớn khung hình. Khi đánh giá YOLO cần đối chiếu ảnh RGB thực tế thay vì giả định model đã thêm luôn xuất hiện trong ảnh.

4.2 Nhiệm vụ

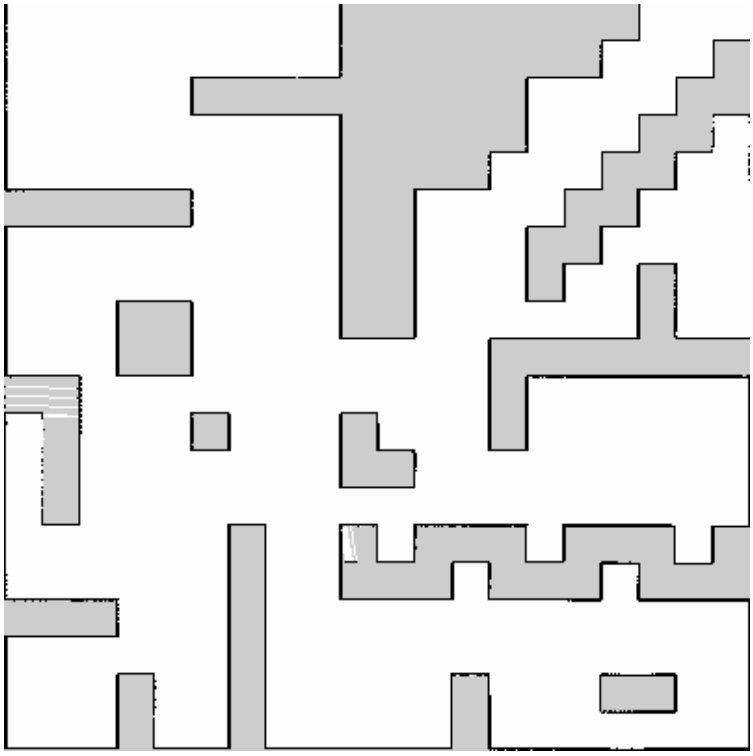
Nhiệm vụ gồm ba phần: định vị trên bản đồ đã có; tự động đi qua ba goal bằng node Python; nhận diện bằng YOLO trong lúc di chuyển hoặc dừng tại goal. Các goal được khảo sát theo vùng trống của map và lưu cố định trong cấu hình để tái hiện lượt chạy.

5 Xây dựng bản đồ bằng SLAM Toolbox

5.1 Nguyên lý và thao tác mapping

Ở tuần 4, SLAM Toolbox kết hợp scan LiDAR với odometry/TF để ước lượng pose và xây occupancy grid 2D khi robot được điều khiển khảo sát. Trong bài tuần 5, em dùng lại hai file map đã lưu, không chạy lại SLAM để tạo map mới. Mục này giữ vị trí trong cấu trúc báo cáo để trình bày nguồn map và mối liên hệ với bài trước.

5.2 Bản đồ đã lưu



Hình 3: *Bản đồ PGM tuần 4 được nạp lại nguyên byte trong bài tuần 5.*

Thuộc tính	Giá trị
Kích thước	401 × 401 ô
Resolution	0,05 m/ô
Origin	$(-10; -10; 0)$
Mode / negate	trinary / 0
Occupied / free threshold	0,65 / 0,25

Bảng 3: *Thông số map từ YAML và PGM.*

Hai file được sao chép từ repository `https://github.com/vanhhhhh512/turtlebot4_bai04` và đối chiếu SHA256 với bản cài từ clone GitHub. Bản đồ thể hiện tường và các cấu trúc đã quét; các model mới của tuần 5 không được ghi vào map tĩnh. Thông tin vật cản mới phụ thuộc sensor và costmap trong thời gian chạy.

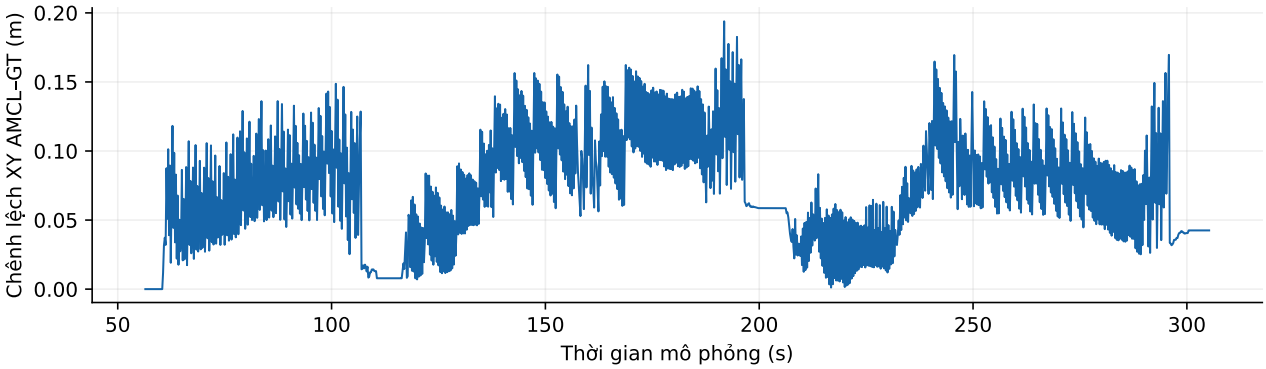
6 Localization sau khi nạp lại bản đồ

6.1 Trình tự khởi tạo

MapServer đọc map.yaml và phát `/map`. AMCL sử dụng bản đồ, scan và chuyển động odometry để cập nhật pose cùng transform *map* → *odom*. Trong lượt chính, action Undock trả thành công trước khi khởi tạo AMCL. Initial Pose được lấy từ Gazebo tại $x = -0,353\,\text{m}; y = -0,000\,\text{m}; yaw = 3,136\,\text{rad}$.

Đây là cách khởi tạo thuận tiện cho mô phỏng, sử dụng ground truth làm đầu vào. Việc pose ban đầu trùng GT không phải phép đánh giá độc lập độ chính xác AMCL. Sau khởi tạo, chương trình vẫn chờ AMCL/Nav2 active và đối chiếu scan với map trên RViz.

6.2 Quan sát localization và đối chiếu log



Hình 4: *Chênh lệch XY giữa pose AMCL và Gazebo được lưu trong cùng mẫu telemetry của run10. Hai pose có thể khác timestamp.*

RViz dùng Fixed Frame = map và hiển thị scan cùng tường để kiểm tra localization. Trong log run10, chênh lệch XY lớn nhất giữa các pose đã lưu là 0,194 m; không có relocalization giữa tuyến. Giá trị này phản ánh hai snapshot có thể khác timestamp, không phải phép đo độc lập sai số AMCL đã đồng bộ thời gian. Trạng thái active chỉ phản ánh node đã kích hoạt; còn cần quan sát scan/map và diễn biến pose.

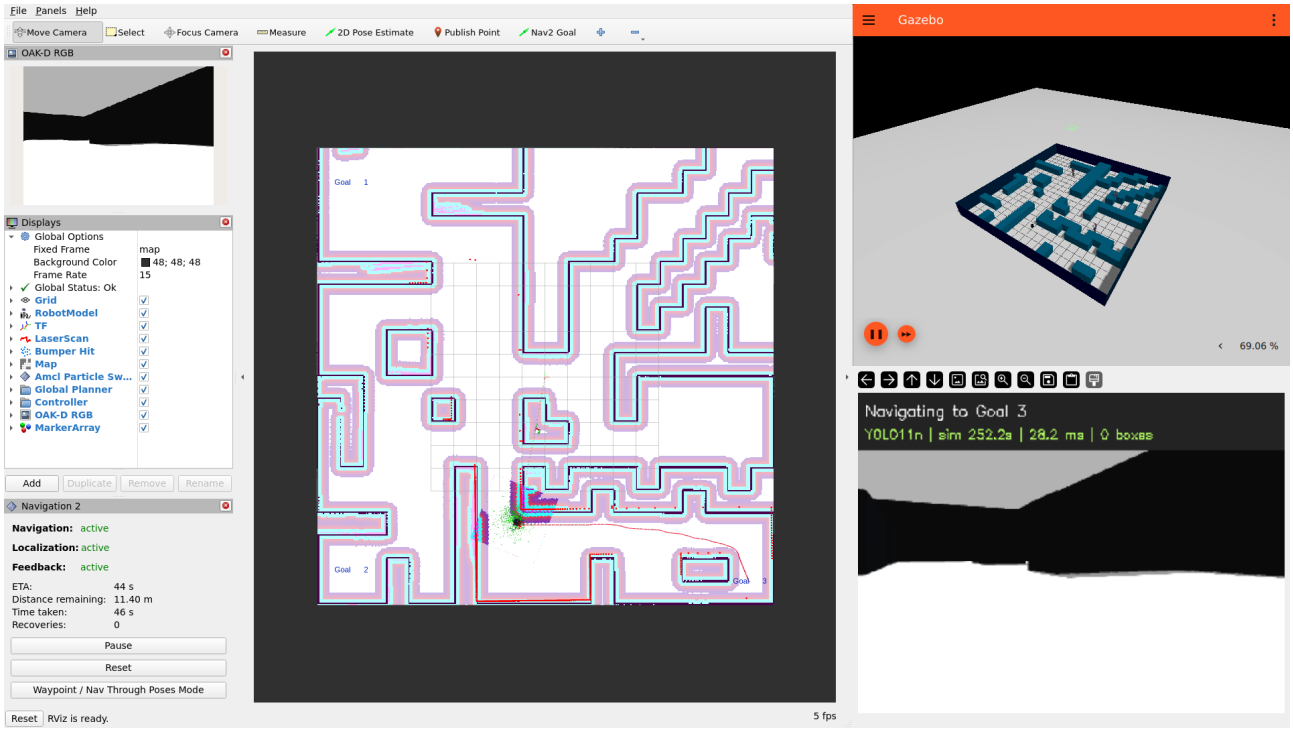
7 Navigation với Nav2

7.1 Lập kế hoạch và điều khiển

Node `bai05_route_navigator` kế thừa BasicNavigator. Hàm `goToPose()` gửi NavigateToPose trong frame map; `isTaskComplete()` cho phép chờ kết quả trong khi callback telemetry vẫn được xử lý. bt_navigator điều phối yêu cầu; planner_server dùng NavfnPlanner để tạo global path; controller_server dùng DWBLocalPlanner cùng local costmap để tính vận tốc. Lệnh được chuyển qua velocity smoother và `/cmd_vel` tới robot.

Cấu hình lưu trong config/nav2.yaml giữ tốc độ thẳng tối đa 0,26 m/s và góc tối đa 1,0 rad/s của controller. Costmap kết hợp bản đồ tĩnh, dữ liệu quan sát và inflation. Trong bài này YOLO chỉ nhận diện và hiển thị; detection RGB chưa được dùng làm input tránh vật cản hay thay đổi goal. Tránh va chạm do Nav2/costmap và sensor của robot đảm nhiệm. Mỗi chặng chỉ có một đích NavigateToPose. Sau Goal 2, chương trình gửi trực tiếp Goal 3; Nav2 có thể cập nhật đường đi khi costmap thay đổi nhưng đích vẫn là Goal 3. Việc YOLO nhìn thấy bình không kích hoạt gửi goal hoặc nối thêm đường đi.

7.2 Ảnh RViz



Hình 5: *RViz hiển thị camera OAK-D RGB ở góc trên trái cùng map, scan LiDAR, costmap và path. Ảnh chụp từ lượt* `corners_run04` *minh họa giao diện có RGB trong RViz; số liệu kết quả cập nhật lấy từ run10 ở mục 9.*

Panel `OAK-DRGB` dùng display Image, nhận topic `/oakd/rgb/preview/image_raw`; camera RGB được hiển thị trực tiếp trong RViz cùng dữ liệu LiDAR/map.

Các dải màu quanh tường thể hiện chi phí vật cản và inflation. Marker Goal 1–3 chỉ hiển thị vị trí trong file cấu hình; marker không phát action hoặc lệnh vận tốc.

7.3 Thực hiện và phân loại goal

Goal	x (m)	y (m)	yaw (rad)	Hệ tọa độ
1	-8,50	8,50	-1,5708	map
2	-8,50	-8,50	0,0000	map
3	9,00	-9,00	1,5708	map

Bảng 4: *Các mục tiêu lưu trong goals.yaml.*

Tọa độ được chọn theo map đã quét và đối chiếu với các khu vực của world. File cấu hình lưu cả yaw; Publish Point chỉ cho vị trí, hướng kết thúc phải được chọn riêng. Chương trình lần lượt gọi goToPose, chờ kết quả, kiểm tra SUCCEEDED và các tín hiệu va chạm rồi mới gửi goal kế tiếp. Không có thao tác Nav2 Goal bằng chuột hoặc teleop trong lượt run10. Log ghi Goal 2 thành công tại 200,760 giây mô phỏng và gửi Goal 3 tại 206,115 giây; giữa hai event chỉ có thời gian dừng, không có goal phụ.

```
for goal in goals:
    pose = make_pose(frame_id='map', x=goal.x, y=goal.y, yaw=goal.yaw)
    navigator.goToPose(pose)
    while not navigator.isTaskComplete():
        check_timeout()
    result = navigator.getResult()
```

```
    save_result_and_telemetry(result)
    if result != TaskResult.SUCCEEDED or collision_detected:
        stop_mission()
```

Đoạn trên tóm tắt thuật toán; mã nguồn thật còn có bước undock, initial pose, ghi log, kiểm tra timeout và thời gian dừng giữa các goal.

7.4 Nhận diện vật thể từ camera bằng YOLO

Node `bai05_yolo_detector` subscribe `/oakd/rgb/preview/image_raw` dưới dạng `sensor_msgs/msg/Image`, gọi `imgmsg_to_cv2()` với `desired_encoding='bgr8'` rồi đưa ảnh vào YOLO-World TorchScript đi kèm source. Mô hình nhận tham số imgsz=960, confidence=0,12 và device GPU. Ảnh camera thực trong lượt chạy có kích thước 320×240; imgsz là cấu hình xử lý của model, không có nghĩa camera cung cấp ảnh 960 pixel gốc. Prompt hình trụ đỏ có dải trắng được ánh xạ thành nhãn `fireextinguisher`; node vẽ/lưu class này cùng `person`.

Từ kết quả suy luận, node lấy class, confidence và box xyxy; vẽ nhãn trong khung ảnh để tránh bị che khi box chạm mép trên. Node publish ảnh `bgr8` và JSON detection có timestamp/frame_id, đồng thời in thông tin ra terminal. Raw frame và ảnh annotate được lưu theo định kỳ khi có detection để đối chiếu kết quả.



Hình 6: *Ảnh camera run10 có bbox person (trái, confidence 0,951) và bình trên tuyến Goal 3 (phải, confidence 0,824). Nhãn là dự đoán của model, không phải ground truth.*

8 Khó khăn gặp phải và cách xử lý

8.1 Bridge sử dụng sai tên world

Lượt chạy đầu dùng đường dẫn file SDF làm biến world của launch upstream. Các bridge vì vậy tạo topic có đoạn `/world//home/...`, khiến ROS báo lỗi remap. Em tách đường dẫn file world khỏi tên thực thể world: Gazebo mở file được sinh khi launch, còn các bridge dùng đúng tên `maze`. Lỗi được giữ lại trong log scout.

8.2 Tham số ROS và đường dẫn plugin mô phỏng

Tham số device truyền bằng chuỗi số 0 bị launch diễn giải thành integer trong khi node khai báo string. Cách sửa là dùng ParameterValue với value_type=str. Tên weights đổi thành yolo_model để không trùng model của TurtleBot. Khi chạy Ignition riêng, cần thêm `/opt/ros/humble/lib` vào system plugin path; nếu thiếu, `libign_ros2_control-system.so` không nạp và controller_manager không xuất hiện.

8.3 Mô phỏng chậm trên màn hình ảo

Màn hình desktop đang khóa nên em dùng Xvfb để bố trí/quay các cửa sổ. Lượt chạy server trên display ảo dùng software rendering và undock bị timeout: log chỉ tiến được khoảng 3,84 giây simulation khi hết thời gian chờ. Em chuyển server cảm biến sang display có GPU, còn GUI/RViz/OpenCV vẫn nằm trên display ảo. Sau thay đổi, robot thực hiện được trọn tuyến. Các lượt lỗi không được tính là demo hoàn thành.

8.4 Giới hạn quan sát và nhận diện

Camera đặt thấp nên khi robot gần người, phần đầu có thể nằm ngoài khung ảnh. Tường làm che khuất model ở một số hướng; YOLO cũng có thể nhầm mặt tường/biên vật thể thành class khác. Em giữ các detection này trong log để nhận xét thay vì lọc bỏ khỏi số liệu tổng hợp. Bài hiện chưa có bộ ảnh gán nhãn để tính precision, recall hoặc mAP, và chưa dùng depth để đo khoảng cách vật thể. Một phiên bản trước có pose quan sát chen giữa Goal 2 và Goal 3, làm đường hiển thị đi tới pose phụ thay vì tới Goal 3. Em bỏ bước đó và chuyển bình lên tuyến đi thực tế; run10 kiểm tra lại toàn bộ ba goal với Nav2 tự xử lý vật cản.

9 Kết quả đạt được

9.1 Mô phỏng, sensor và map

Hạng mục	Kết quả
Map	401×401 ô; 0,05 m/ô; origin $(-10; -10; 0)$; SHA256 khớp nguồn tuần 4.
Localization	AMCL active; Fixed Frame map và scan hiển thị cùng tường trên RViz.
Navigation	3/3 action SUCCEEDED; không có BUMP hazard trong cửa sổ chạy route.
Camera RGB	320×240; subscribe bằng Image và chuyển BGR qua cv_bridge.
Object detection	YOLO-World chạy đồng thời; có ảnh/JSON/log person và fire extinguisher.

Bảng 5: *Kết quả chính của lượt chạy tuần 5.*

9.2 Kết quả từng navigation goal

Goal	Action	AMCL (m)	T_{sim} (s)	T_{wall} (s)	e_{goal} (m)
1	SUCCEEDED	11,79	50,65	59,09	0,227
2	SUCCEEDED	20,30	84,46	97,48	0,229
3	SUCCEEDED	22,78	94,51	113,50	0,239

Bảng 6: *Kết quả ba goal theo thứ tự chương trình gửi.*

Tổng quãng đường AMCL của ba lượt là 54,88 m. Quãng đường được tính trên chuỗi pose AMCL lấy mẫu mỗi 0,2 giây mô phỏng; mẫu không đổi không làm tăng tổng:

$$L = \sum_{i=1}^{n} \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2}, \qquad e_{goal} = \sqrt{(x_f - x_g)^2 + (y_f - y_g)^2}.$$

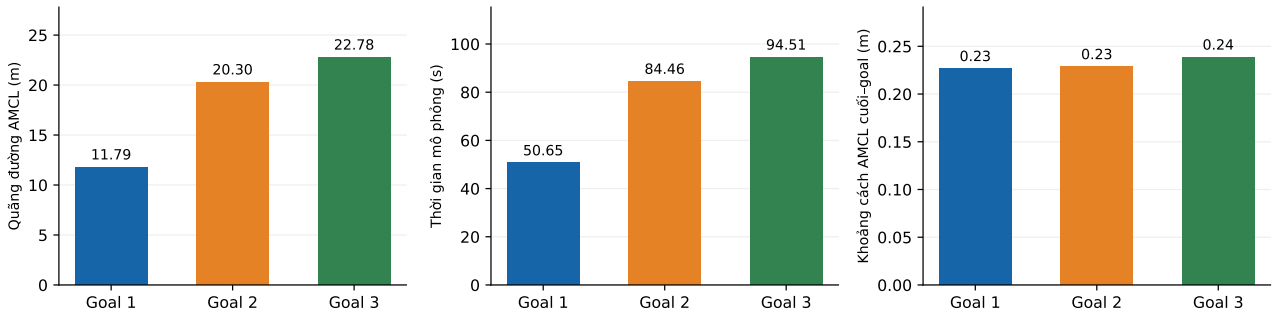
Thời gian trong bảng đo từ trước goToPose đến sau khi nhận result và spin thêm 0,5 giây wall để lấy snapshot. Vì vậy nó gồm khoảng chờ phản hồi và ổn định ngắn, không chỉ là thời gian robot chuyển động. T_{sim} và T_{wall} được giữ riêng.

Sai lệch e_{goal} là khoảng cách từ mẫu AMCL cuối tới tọa độ đích, không phải sai số localization so với vị trí thật. AMCL và GT cuối có độ lệch timestamp 0,408–1,026 s; snapshot AMCL có thể sớm hơn

thời điểm goal checker xác nhận. SUCCEEDED phản ánh điều kiện goal checker của Nav2. Không suy ra độ chính xác toàn hệ hoặc sai số yaw từ một mẫu pose cũ.

Các lượt có 15727 message hazard được xử lý đến cuối Goal 3 và không có detection loại BUMP. Không ghi nhận message contact có dữ liệu. Topic bumper chỉ phát khi có contact; số message bằng 0 cần được đọc cùng video và log hazard, không phải phép chứng minh độc lập mọi loại va chạm. Log `MISSION_COMPLETE` ghi 3 goal, 0 contact và 0 bump.

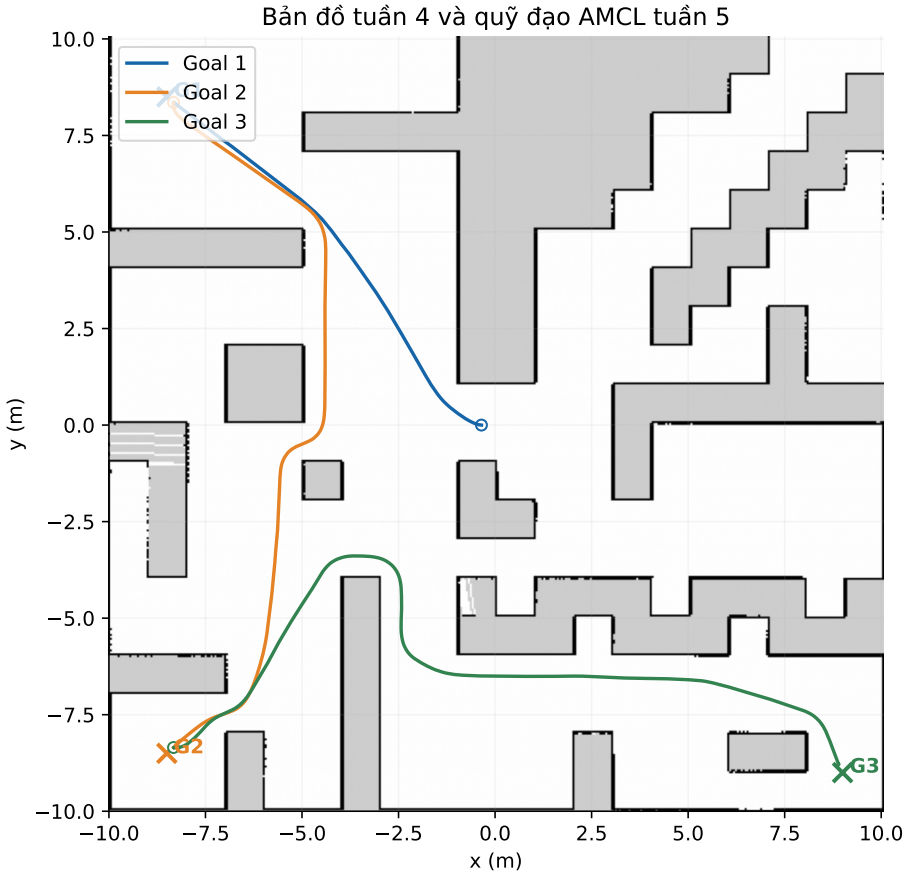
9.3 Quãng đường, thời gian và sai lệch



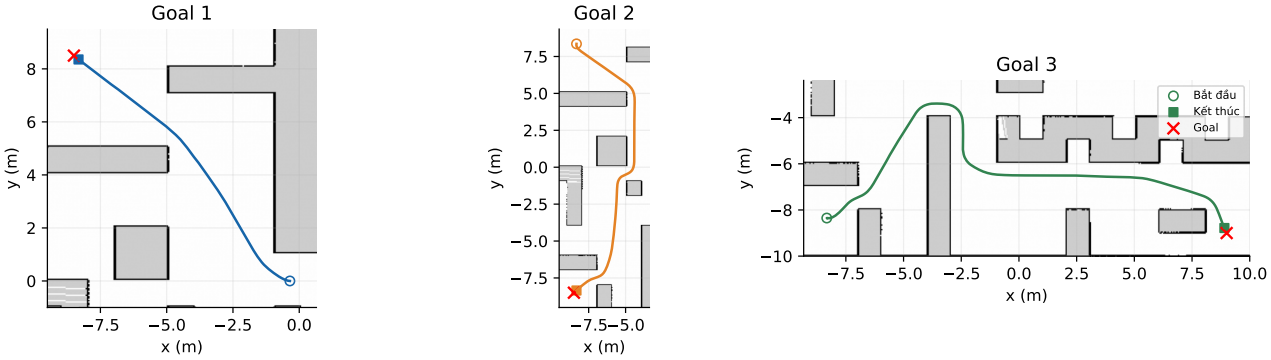
Hình 7: *Quãng đường AMCL, thời gian mô phỏng và khoảng cách pose cuối–goal từ dữ liệu của ba goal.*

Goal 1 có quãng đường ngắn nhất, 11,79 m; Goal 3 dài nhất, 22,78 m. Thời gian còn phụ thuộc góc rẽ, điều khiển và tốc độ mô phỏng. Em giữ cả số đo thực và biểu đồ để đối chiếu thay vì kết luận chỉ từ ảnh robot ở đích.

9.4 Quỹ đạo AMCL theo từng goal

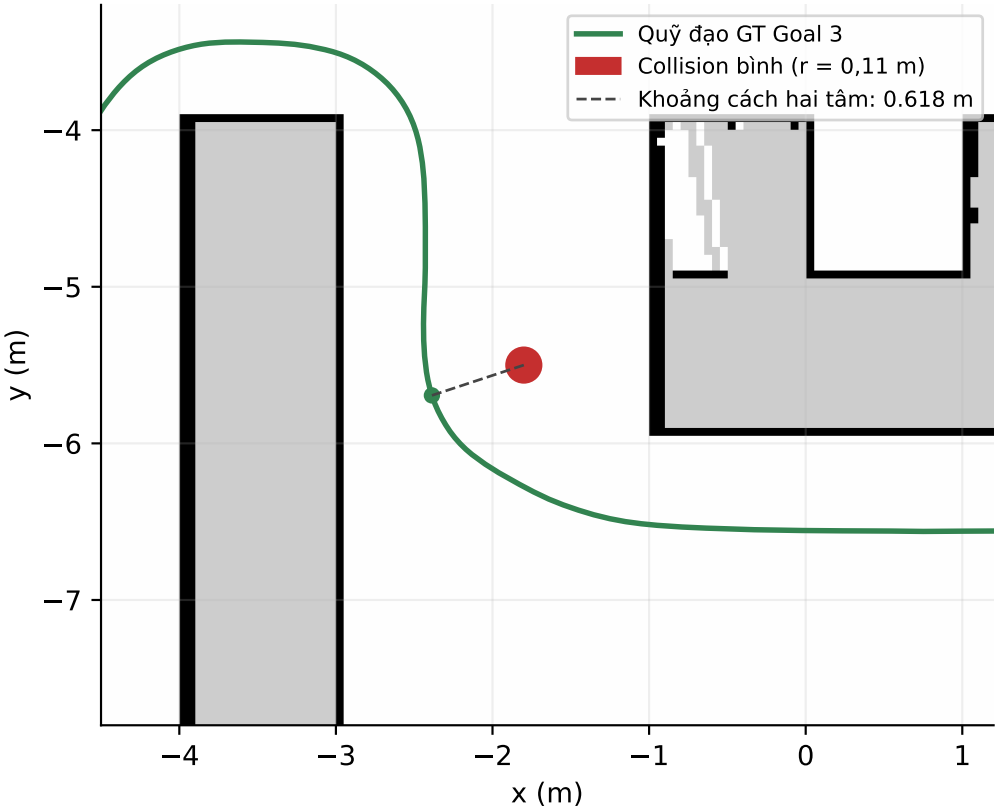


Hình 8: *Map và toàn bộ quỹ đạo AMCL tuần 5; dấu nhân là goal, vòng tròn là điểm đầu từng đoạn.*



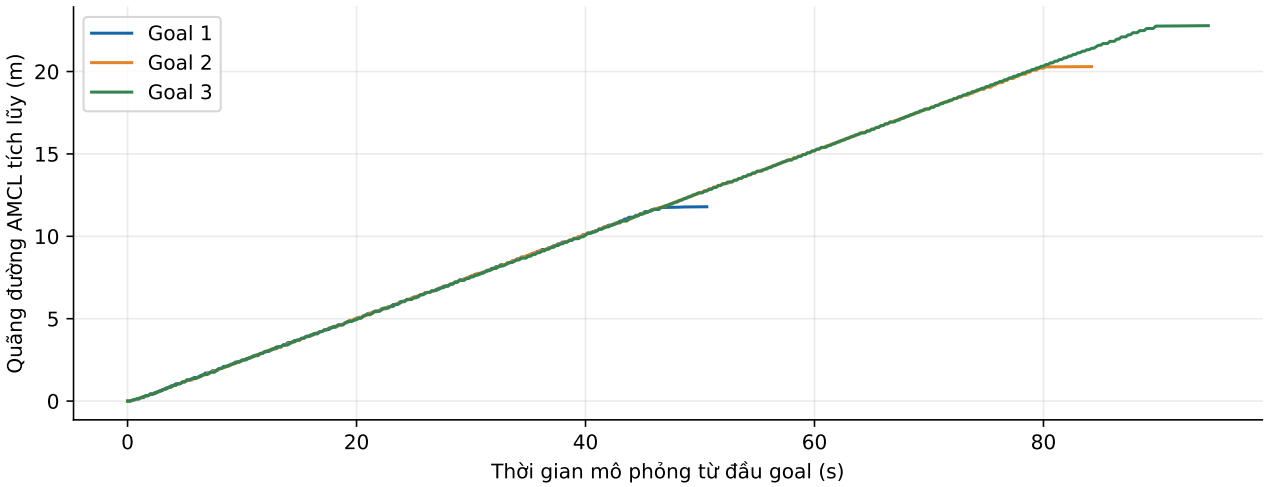
Hình 9: *Quỹ đạo AMCL tách theo goal; vòng tròn là điểm đầu, vuông là điểm cuối, dấu nhân là tọa độ đích.*

Các đường thể hiện pose AMCL đã ghi nhận, không phải global path do planner xuất. Đoạn đi vòng hoặc các góc rẽ phản ánh bố trí hành lang của maze và lựa chọn của controller.



Hình 10: *Quỹ đạo Gazebo khi đi Goal 3 quanh bình tại* $(-1,8; -5,5)$*. Khoảng cách nhỏ nhất giữa tâm robot và tâm bình là 0,618 m; đây không phải khoảng hở giữa hai bề mặt.*

9.5 Quãng đường tích lũy theo thời gian



Hình 11: *Tổng độ dời XY liên tiếp của AMCL theo thời gian mô phỏng trong từng goal.*

Đoạn gần ngang cho thấy vị trí AMCL ít thay đổi; chỉ từ biểu đồ không phân biệt được robot đang quay tại chỗ, đứng chờ hoặc AMCL chưa phát pose mới. Không cộng quãng đường giữa các goal vào một đoạn đo nếu mẫu thuộc thời gian dừng giữa hai goal.

9.6 Kết quả nhận diện và đánh giá

Chỉ số	Kết quả
Frame ảnh đã xử lý	1176
Frame có ít nhất một box	282
Box person trên các frame	132
Box fire extinguisher trên các frame	160
cv_bridge + predict, median	60,86 ms
cv_bridge + predict, p95	81,75 ms

Bảng 7: *Thống kê YOLO trong cửa sổ ghi log của lượt chính.*

Log ghi nhận cả person và fire extinguisher từ camera RGB. Confidence person cao nhất là 0,951, tại giây mô phỏng 72,96. Bình trên tuyến Goal 3 cũng có ảnh raw và ảnh có box lưu riêng; hình minh họa lấy trực tiếp từ camera của lượt này. Scene chứa một người và hai bình, nhưng các số box là tổng qua nhiều frame. Số box cộng qua các frame không phải số người duy nhất: cùng một người có thể được phát hiện nhiều lần. Các chỉ số thời gian chỉ tính chuyển ảnh và predict, chưa tính annotate, publish, vẽ cửa sổ hoặc lưu JPEG; không chuyển trực tiếp nghịch đảo giá trị này thành FPS end-to-end. Chưa có nhãn ground truth cho toàn bộ ảnh nên báo cáo không đưa accuracy hoặc mAP.

10 Cách chạy code

10.1 Chạy all bằng một lệnh

Sau khi cài dependency và build theo README, chạy:

```
cd ~/turtlebot5_github_ws/src/bai05_turtlebot4
./scripts/run_demo.sh
```

Bước	Thao tác	Kết quả mong đợi
1	Chạy script run_demo.sh.	Build các package cần thiết; mở Gazebo, RViz và YOLO.
2	Chờ bootstrap, quan sát pose/scan.	Robot undock; Initial Pose được đặt; Nav2 active.
3	Quan sát chương trình tự chạy route.	Goal1, Goal2, Goal3 được gửi tuần tự bằng code.
4	Quan sát cửa sổ camera và terminal.	Box/class/confidence, JSON và log nhận diện.
5	Chờ MISSION_COMPLETE, xem output_dir.	Có kết quả goal, CSV trajectory và ảnh detection.
6	Ctrl+C terminal launch.	Dừng mô phỏng và các node của phiên.

Bảng 8: *Trình tự thao tác tái hiện chương trình.*

Có thể chọn `device:=cpu` hoặc truyền weights bằng `yolo_model:=...`. Lượt run10 dùng desktop DISPLAY=:0; script căn ba cửa sổ và tự chọn ROS domain, Ignition partition riêng cho mỗi phiên. Mặc định dữ liệu lưu ở `~/bai05_runs/`; có thể đặt `BAI05_OUTPUT_DIR` để chọn thư mục mới cho một lượt.

Source public: `https://github.com/vanhhhhh512/turtlebot4_bai05`. Source map/localization tuần 4: `https://github.com/vanhhhhh512/turtlebot4_bai04`.

Video demo: `https://drive.google.com/drive/folders/1GKdy0U0tKGbxUAaGAmAffTPxJa5pT8wy?usp=drive_link`. Bản code kiểm chứng: commit `c87263f`; số liệu chính: `direct_goal_route_run10`. Báo cáo và biểu đồ được tái tạo bằng script đi kèm hồ sơ; README repository chứa lệnh cài đặt/build và cách chạy riêng từng node.

11 Kiến thức và kỹ năng đã học được

11.1 Phân biệt mapping, localization và navigation

Mapping tạo bản đồ từ sensor và chuyển động; localization ước lượng pose trên map đã có; navigation dùng pose và costmap để tìm đường rồi điều khiển robot tới goal. Ở tuần này em dùng map tuần trước cho AMCL và tập trung lập trình phần gửi goal. Việc có model YOLO nhận diện được người không tự bổ sung người đó vào costmap.

11.2 Làm việc với hệ tọa độ và thời gian ROS 2

Pose mục tiêu cần đúng frame và quaternion. Clock mô phỏng khác thời gian thực, nên đo riêng hai loại thời gian và lưu timestamp của từng nguồn. Khi gặp lỗi TF, cần kiểm tra node phát transform, thời điểm dữ liệu và trình tự khởi động; trạng thái hiển thị RViz chỉ là một phần của quá trình kiểm tra.

11.3 Nhận biết giới hạn của sensor

LiDAR 2D quan sát mặt phẳng quét, còn RGB phụ thuộc góc nhìn và vật thể có nằm trong ảnh hay không. Camera thấp có thể chỉ nhìn thấy một phần người; tường hoặc vật cản che khuất làm detector bỏ sót. Confidence là đầu ra mô hình cho từng box, không phải bằng chứng chắc chắn class đúng và không phải xác suất hệ thống an toàn.

11.4 Lập trình node và đánh giá kết quả

Em nắm được cách dùng action để chờ một nhiệm vụ dài, chuyển trạng thái tuần tự và dừng khi có lỗi; dùng subscriber/timer để tách nhận ảnh và suy luận; chuyển Image message sang OpenCV bằng cv_bridge. Qua log và biểu đồ, em cũng thấy cần giải thích rõ nguồn dữ liệu, cách tính và trường hợp chưa đo được trước khi kết luận.

Tài liệu tham khảo

1. TurtleBot 4 User Manual – Navigator và Simulation.
   `https://turtlebot.github.io/turtlebot4-user-manual/tutorials/turtlebot4_navigator.html`
2. Nav2 Simple Commander API.
   `https://docs.nav2.org/commander_api/index.html`
3. Ultralytics – Model Prediction.
   `https://docs.ultralytics.com/modes/predict/`
4. vision_opencv / cv_bridge.
   `https://github.com/ros-perception/vision_opencv/tree/humble/cv_bridge`
5. Gazebo Fuel – MaleVisitorOnPhone v2, tác giả Wan Yi Seow, CC BY 4.0. Metadata được giữ trong repository; bình chữa cháy được dựng bằng SDF trong source bài tập.